Guía rápida de Git y GitHub

Para trabajar en pareja en el proyecto de ajuste de trazas con filtro de Kalman

1. Las ideas básicas

Git guarda la historia de tu proyecto como una serie de «fotos» (commits). GitHub es la web donde se guarda una copia de esa historia para compartirla. Estos son los conceptos que necesitas:

Concepto	Qué es, en pocas palabras
Repositorio (repo)	La carpeta del proyecto más toda su historia (guardada en la carpeta oculta `.git`).
Commit	Una foto del proyecto en un momento dado, con un mensaje que explica qué cambió.
Branch (rama)	Una línea de trabajo paralela. Es una copia **completa** del proyecto donde haces cambios sin tocar `main`.
main	La rama principal. Debe contener siempre una versión que funcione.
origin	El nombre que Git da a la copia del repo que está en GitHub.
push / pull	*push*: subir tus commits a GitHub. *pull*: bajarte los commits nuevos de GitHub.
Merge	Juntar los cambios de una rama en otra.
Pull Request (PR)	Petición en GitHub para hacer merge de tu rama en `main`, para que el otro la revise antes.

2. Configuración inicial (solo una vez)

Dile a Git quién eres. Usa el mismo email que tu cuenta de GitHub:

```
git config --global user.name "Alvaro Martinez"
git config --global user.email "tu@email.com"
```

Clona el repo en una carpeta **fuera de OneDrive** y entra en ella:

```
cd C:\Users\alvar\Proyectos
git clone https://github.com/tu-usuario/nombre-del-repo.git
cd nombre-del-repo
```

Crea el entorno del proyecto (con Miniforge funciona igual que con Anaconda):

```
conda env create -f environment.yml
conda activate env_py312_kalman
```

Cuidado. OneDrive sincroniza la carpeta `.git` mientras Git la modifica, y eso puede bloquear o duplicar archivos. GitHub ya hace de copia de seguridad, así que no hace falta OneDrive.

- **Contraseña:** GitHub no acepta tu contraseña normal al hacer push. Usa un *token personal* (Settings → Developer settings → Personal access tokens) o configura una clave SSH.
- **Terminal:** en Windows, **Git Bash** acepta los comandos de Linux (`ls`, `ll`, `cat`). En PowerShell usa `ls -Force` para ver archivos ocultos. Los comandos `git` funcionan igual en todas.
- **Añadir a tu compañero:** en el repo, Settings → Collaborators → Add people. Tiene que aceptar la invitación por email.

3. El ciclo básico: cambiar, guardar, subir

Este es el ciclo que repetirás constantemente:

```
git status                        # ¿qué ha cambiado?
git add archivo.py                # preparar un archivo para el commit
git add .                         # ... o preparar todos los cambios
git commit -m "Añade predicción del filtro de Kalman"
git push                          # subir a GitHub
```

Y para bajarte lo que haya subido tu compañero:

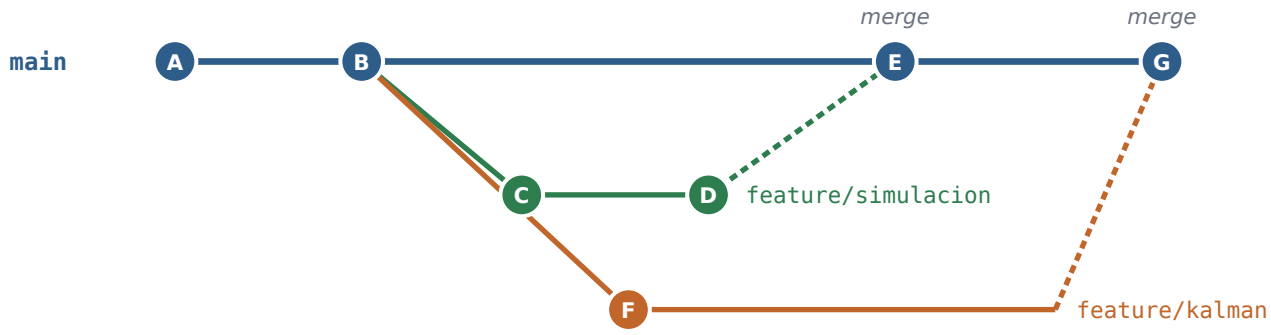
```
git pull
```

Cómo escribir buenos commits

- **Pequeños y frecuentes:** un commit por cada cambio con sentido propio (una función nueva, un bug arreglado). No un commit gigante al final del día.
- **Mensajes claros, en imperativo:** «Añade matriz de covarianza del ruido», «Corrige signo en la actualización». Evita «cambios», «cosas» o «arreglos».
- **Revisa antes de confirmar:** `git status` y `git diff` te muestran exactamente qué vas a guardar.

4. Las branches: cómo funcionan

La idea clave: **una branch no es «una parte» del proyecto**. Cuando creas una rama a partir de `main`, contiene **todo** lo que había en `main` en ese momento. Tú vas añadiendo tus cambios encima.



- **A** y **B** son commits en `main`.
- Desde **B** salen dos ramas. Cada una tiene una copia completa del proyecto hasta **B**.
- Tú trabajas en `feature/simulacion` (commits **C**, **D**) y tu compañero en `feature/kalman` (commit **F**).
- Cuando la simulación está lista, se fusiona en `main` → **E**. Después se fusiona el Kalman → **G**, y `main` tiene las dos cosas.

Mientras trabajáis en paralelo, **cada uno no ve los cambios del otro** hasta que se juntan en `main`. Eso es justo lo que permite trabajar sin pisaros.

Comandos de ramas

```
git branch                          # ver ramas locales (* = la actual)
git checkout -b feature/simulacion # crear una rama nueva y pasarte a ella
git checkout main                   # volver a main
git push -u origin feature/simulacion   # subir la rama a GitHub (la 1ª vez)
git branch -d feature/simulacion   # borrar la rama local (ya fusionada)
```

Consejo. Antes de crear una rama nueva, ponte en `main` y haz `git pull`. Así partes de la versión más reciente.

Cómo dividir el trabajo del Kalman en ramas

Lo ideal es que cada rama corresponda a **una tarea concreta** y que toque archivos distintos a los de tu compañero. Por ejemplo:

Rama	Qué contiene	Archivos
`feature/geometria`	Planos del detector, campo magnético	src/kalmanfit/geometry.py
`feature/simulacion`	Generar trazas con ruido y dispersión múltiple	src/kalmanfit/simulation.py
`feature/kalman-filter`	Predicción y actualización del filtro	src/kalmanfit/kalman.py
`feature/smoother`	Suavizado hacia atrás	src/kalmanfit/kalman.py
`feature/analisis`	Residuos, pulls, distribución de χ^2	src/kalmanfit/analysis.py
`fix/covarianza`	Arreglar un error concreto	el que toque

- **Ramas cortas:** mejor varias ramas de pocos días que una enorme de un mes. Cuanto más tiempo vive una rama, más difícil es fusionarla.

- **Nombres con prefijo:** `feature/` para cosas nuevas, `fix/` para arreglos, `docs/` para README o informe.
- **Nunca trabajes directamente en main.** Todo entra por Pull Request.

5. Merges y Pull Requests

Un **Pull Request** es la forma ordenada de llevar tu rama a `main`: GitHub muestra los cambios, tu compañero los revisa, y luego se fusionan con un botón.

El flujo completo, paso a paso

1	**Parte de main actualizado**	`git checkout main` `git pull`
2	**Crea tu rama**	`git checkout -b feature/simulacion`
3	**Trabaja y haz commits**	`git add .` `git commit -m "Añade generador de trazas rectas"`
4	**Sube la rama**	`git push -u origin feature/simulacion`
5	**Abre el PR en GitHub**	Aparece el botón «Compare & pull request». Escribe qué hace el cambio.
6	**Revisión**	Tu compañero mira los cambios y comenta si algo no cuadra. Puedes seguir haciendo commits y push en la misma rama: el PR se actualiza solo.
7	**Merge**	Se pulsa «Merge pull request». Tus cambios ya están en main.

8	**Limpia**	`git checkout main`
		`git pull`
		`git branch -d feature/simulacion`

Traer a tu rama lo que ya está en main

Si tu compañero ya ha fusionado algo que tú necesitas (por ejemplo, la geometría del detector para tu filtro), lo incorporas a tu rama así:

```
git checkout main
git pull                      # bajar lo último de GitHub
git checkout feature/kalman-filter
git merge main                # traer esos cambios a tu rama
```

Haz esto a menudo si tu rama dura varios días: así los conflictos son pequeños y fáciles de resolver.

6. Conflictos

Un conflicto aparece cuando **los dos habéis cambiado las mismas líneas del mismo archivo**. Git no sabe cuál elegir y te lo marca así:

```
<<<<<<< HEAD
Q = sigma_ms**2 * np.eye(2)
=======
Q = theta0**2 * np.array([[1, 0], [0, 1]])
>>>>>>> main
```

- Arriba (entre <<<<<<< y =======) está tu versión; abajo, la otra.
- Edita el archivo dejando la versión correcta (o una mezcla) y **borra las tres líneas de marcas**.
- Luego: `git add archivo.py` y `git commit`.
- VS Code muestra botones «Accept Current / Incoming / Both» que lo hacen más cómodo.

> **Consejo.** La mejor forma de evitar conflictos es repartirse el trabajo por archivos o funciones, hacer ramas cortas y traer `main` a tu rama con frecuencia.

7. Deshacer errores

Situación	Comando
Ver la historia de commits	`git log --oneline --graph`
Descartar cambios no guardados en un archivo	`git restore archivo.py`
Sacar un archivo del *add* (sin perder cambios)	`git restore --staged archivo.py`
Cambiar el mensaje del último commit (aún sin push)	`git commit --amend -m "Nuevo mensaje"`
Deshacer un commit que ya está en GitHub	`git revert <id_del_commit>`
Dejar de seguir un archivo que ya subiste (y meterlo en .gitignore)	`git rm --cached archivo`
Guardar cambios a medias para cambiar de rama	`git stash / git stash pop`

> **Cuidado.** Evita `git reset --hard` y `git push --force` salvo que sepas muy bien lo que haces: pueden borrar trabajo, incluido el de tu compañero. Para deshacer algo ya subido, usa siempre `git revert`, que crea un commit nuevo que anula el anterior.

8. Archivos importantes del repo

Archivo	Para qué sirve
`.gitignore`	Lista de archivos que Git no debe subir: cachés de Python, datos generados, figuras temporales, archivos auxiliares de LaTeX.
`README.md`	Portada del repo: qué es el proyecto, cómo instalarlo, cómo ejecutarlo y cómo está organizado.
`environment.yml`	Entorno de conda/Miniforge (incluye la versión de Python). Se crea con `conda env create -f environment.yml`.
`requirements.txt`	Lo mismo para quien use pip: `pip install -r requirements.txt`.

- Cuando alguien añada una librería nueva, la apunta **en los dos archivos de dependencias en el mismo commit**.
- Si el `environment.yml` cambia, el otro actualiza su entorno con `conda env update -f environment.yml`.
- No subáis datos pesados ni resultados: se regeneran con los scripts.

Estructura sugerida

```
kalman-fit/
├── README.md
├── environment.yml
├── requirements.txt
├── .gitignore
├── src/kalmanfit/     geometry.py, simulation.py, kalman.py, analysis.py
├── tests/             comprobaciones (p. ej. traza sin ruido -> ajuste exacto)
├── notebooks/         exploración y gráficas
├── scripts/           scripts que generan los resultados finales
└── report/            informe en LaTeX y figuras
```

9. Chuleta de comandos

Comando	Qué hace
`git clone <url>`	Descargar el repo por primera vez
`git status`	Ver qué ha cambiado y en qué rama estás
`git diff`	Ver los cambios línea a línea
`git add <archivo> / git add .`	Preparar cambios para el commit
`git commit -m "mensaje"`	Guardar una foto con los cambios preparados
`git push`	Subir commits a GitHub
`git pull`	Bajar commits nuevos de GitHub
`git branch`	Listar ramas
`git checkout -b <rama>`	Crear una rama y pasarte a ella
`git checkout <rama>`	Cambiar de rama
`git merge <rama>`	Traer los cambios de otra rama a la actual
`git log --oneline --graph`	Ver la historia de forma compacta
`git remote -v`	Comprobar a qué repo de GitHub está conectado
`git stash / git stash pop`	Aparcar cambios a medias y recuperarlos

10. Reglas de oro para trabajar en pareja

- **main siempre funciona.** Nada entra en main sin Pull Request. Podéis protegerla en Settings → Branches → Add rule.
- **git pull antes de empezar** a trabajar cada día.
- **Una rama por tarea**, con nombre claro y vida corta.
- **Repartíos archivos o funciones distintas** para no pisaros.
- **Commits pequeños** con mensajes que expliquen qué cambia.
- **Revisad los PR del otro**: es la forma de que los dos entendáis todo el código (y os preguntarán por él en la defensa).
- **Comunicaos:** antes de empezar una rama, decid qué vais a tocar.
- **Si algo se rompe, no hagáis force push.** Pedid ayuda o usad `git revert`.